

CodeFab SW Design

Interpreter Pipeline, Parser Registry, AST Visitor 구조를 중심으로 보는 현재 설계 발표 자료

[Overview](#)[Pipeline](#)[Parser](#)[AST](#)[Visitor](#)[Runtime](#)[Patterns](#)[Limits](#)[Roadn](#)

1. 발표 목적

CodeFab는 팀 전용 Custom Language를 실행하기 위한 C++ 기반 인터프리터입니다. 이 발표는 현재 코드베이스의 구조를 SW 설계 관점에

서 설명하고, 적용된 디자인 패턴과 클래스 간 책임 분리를 정리합니다.

Pipeline

Tokenizer, Parser, Checker, Executor
로 이어지는 단계적 실행 구조입니
다.

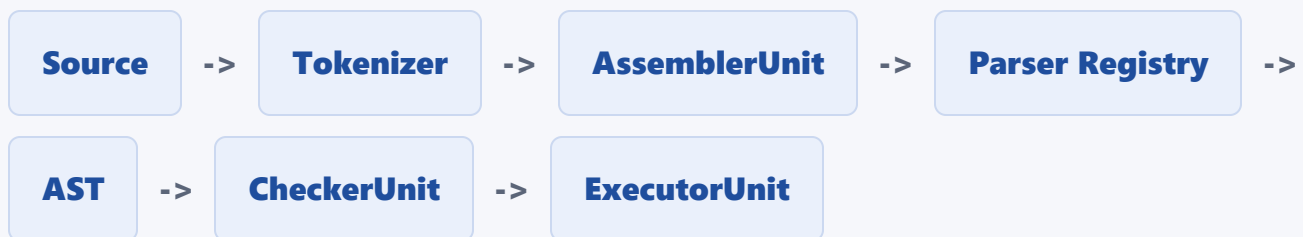
Extensible Parser

StatementParserRegistry와
IStatementParser로 문장 parser를
확장합니다.

Visitor Pass

CheckerUnit과 ExecutorUnit은 같은
AST를 서로 다른 목적의 pass로 순
회합니다.

2. 전체 실행 파이프라인



Tokenizer

입력 문자열을 TokenList로 변환합니다. import 문을 만나면 대상 파일을 재귀적으로 splice
합니다.

AssemblerUnit

TokenList를 순회하며 ProgramNode AST를 조립합니다.

Registry

현재 TokenType에 맞는 IStatementParser 구현체를 선택합니다.

CheckerUnit

AST 의미 검사와 정적 정보 기록을 수행합니다.

ExecutorUnit

AST를 실행하며 runtime scope와 value를 관리합니다.

2.1 진입점: ShellMode 전략

main()은 argv를 보고 ShellMode 구현체 하나를 선택합니다. 각 모드는 자신만의 파이프라인을 Run() 안에서 지역으로 소유합니다.

ShellMode 구현체	용도	상태
PromptMode	REPL(한 줄씩 입력)	구현됨
FileMode	run <file> — 파일 전체 실행	구현됨
DebugMode	debug <file> — step/break/watch 등	스텝 — 미구현 메시지만 출력

3. 주요 모듈 책임

모듈	책임
Tokenizer	문자열 입력을 토큰 단위로 분해
AssemblerUnit	토큰 스트림을 statement 단위로 AST에 조립
StatementParserRegistry	TokenType에 맞는 parser factory 관리
ExpressionParser	연산자 우선순위 기반 expression parsing
SyntaxNode	AST 노드 공통 기반 클래스
NodeVisitor	AST 순회 동작의 공통 인터페이스
CheckerUnit	의미 검사, 정적 바인딩, 상수 폴딩
ExecutorUnit	AST 실행, runtime value/scope/function/class 처리

4. Parser Architecture

Parser 계층은 AssemblerUnit과 StatementParserRegistry를 중심으로 동작합니다. AssemblerUnit은 구체 parser를 직접 알지 않고, 현재 토큰 타입을 registry에 전달해 parser를 선택합니다.

```
AssemblerUnit
```

```
-> tokenList[pos].type 확인
```

```
-> StatementParserRegistry::Resolve(tokenType)
-> IStatementParser::Parse(tokenList, pos)
-> SyntaxNode 반환
```

Registry Pattern

Factory Pattern

Strategy Pattern

등록된 statement parser: VarDeclareParser, PrintStatementParser, IfStatementParser, ForStmtParser, BlockParser, FunctionStatementParser, ReturnStatementParser, ClassStatementParser, ImportStatementParser, ExpressionParser.

장점

- 문장 종류별 parser 분리
- 새 문법 추가 시 변경 범위 축소
- AssemblerUnit의 parser 의존성 감소

주의점

- ExpressionParser가 expression statement 역할도 수행
- 세미콜론 소비 책임이 parser마다 다름
- 전역 singleton registry는 테스트 오염 가능성 존재

5. AST Design

AST는 SyntaxNode 추상 클래스와 구체 노드 계층으로 구성됩니다. 모든 노드는 children을 통해 트리 구조를 만들며, Accept()로 Visitor를 받아들이습니다.

SyntaxNode

- NodeType type
- Token token
- vector<unique_ptr<SyntaxNode>> children
- Accept(NodeVisitor&)

노드	children 의미
BinaryExprNode	[left, right]
AssignExprNode	[target, value]
IfStmtNode	[condition, thenBranch, optionalElseBranch]
ForStmtNode	[init, condition, increment, body]
FuncDeclStmtNode	[param..., body]
ClassDeclStmtNode	[method(FuncDeclStmtNode)...] (+ parentNameToken)
ImportStmtNode	[path(StringLiteralNode), alias(IdentifierNode)]
MemberAccessExprNode	[target] (+ token = 멤버 이름)

6. Visitor 기반 Pass 구조

CheckerUnit과 ExecutorUnit은 NodeVisitor를 상속합니다. 각 SyntaxNode는 Accept()에서 자기 타입에 맞는 Visit()을 호출합니다.

```
node->Accept(visitor)
-> visitor.Visit(concreteNode)
```

CheckerUnit

- 중복 선언 검사
- return 위치 검사
- this/super 위치 검사
- 정적 바인딩 거리 기록
- 상수 폴딩 결과 기록

ExecutorUnit

- statement 실행
- expression 평가
- runtime scope 관리
- 함수/배열/클래스 실행
- 출력과 런타임 에러 처리

7. Runtime Model

ExecutorUnit은 Value_t 기반으로 runtime 값을 표현합니다. 배열과 인스턴스는 shared_ptr로 관리되어 변수 대입 후에도 같은 객체를 공유합니다.

```
Value_t =
    double
    string
    bool
    shared_ptr<FunctionObject>
    monostate
    shared_ptr<ArrayObject>
    shared_ptr<InstanceObject>
```

ArrayObject

배열 값은 shared_ptr로 전달되어 동일 저장소를 공유합니다.

InstanceObject

클래스 인스턴스 필드는 동적 map으로 관리됩니다.

FunctionObject

함수 파라미터와 body AST 포인터를 보관합니다.

8. 적용된 디자인 패턴

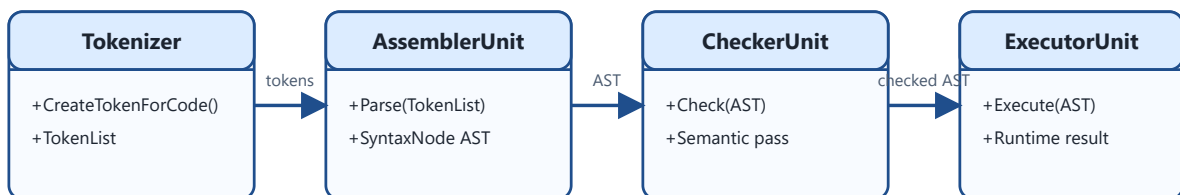
패턴	적용 위치	설명
Pipeline	전체 구조	Tokenize -> Parse -> Check -> Execute
Registry	StatementParserRegistry	TokenType별 parser factory 등록/조회

패턴	적용 위치	설명
Factory	StatementParserRegistrar	parser 객체 생성 책임 캡슐화
Strategy	IStatementParser 구현체	statement 종류별 parsing strategy
Composite	SyntaxNode::children	AST를 트리 구조로 표현
Visitor	NodeVisitor	AST 노드별 동작 분리
Singleton	StatementParserRegistry::Instance()	parser registry 전역 접근

패턴별 간략 class diagram

Pipeline

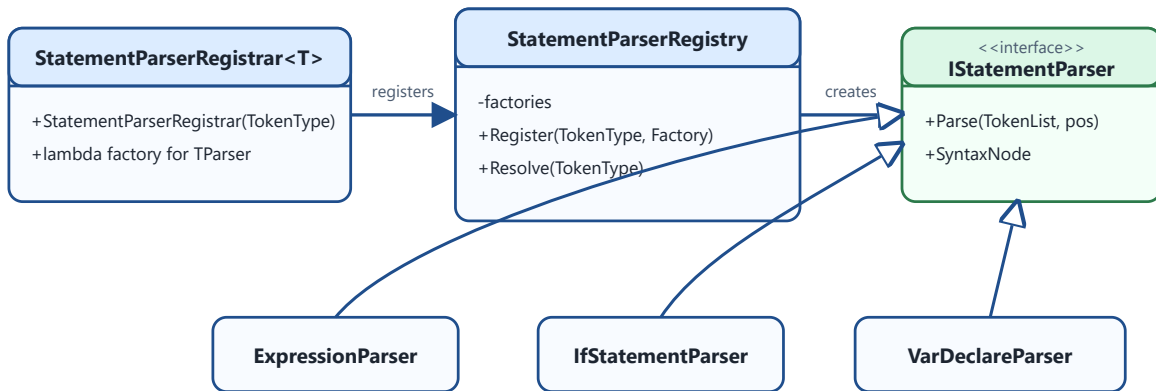
Pipeline Pattern



각 단계가 이전 단계의 산출물을 입력으로 받습니다.

Registry + Factory

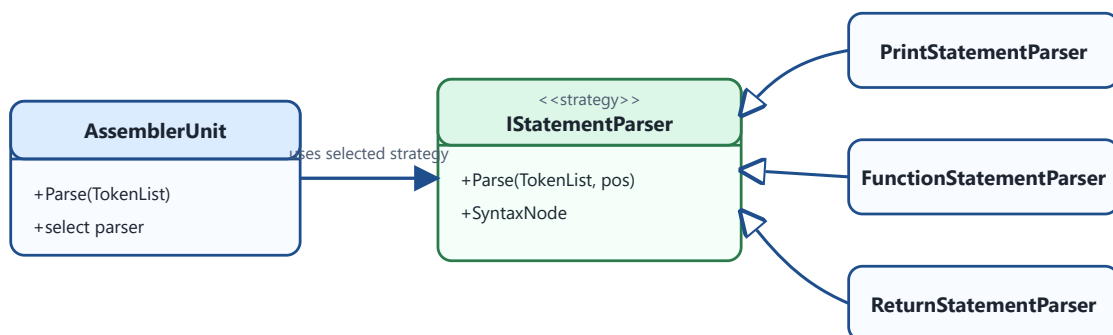
Registry + Factory Pattern



TokenType별 parser factory를 등록하고 조회합니다.

Strategy

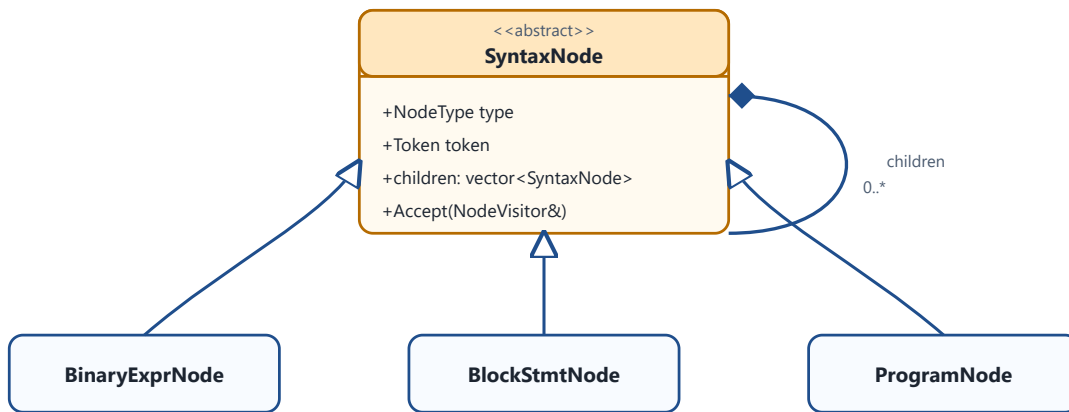
Strategy Pattern



문장 종류별 parsing algorithm을 parser 클래스로 분리합니다.

Composite

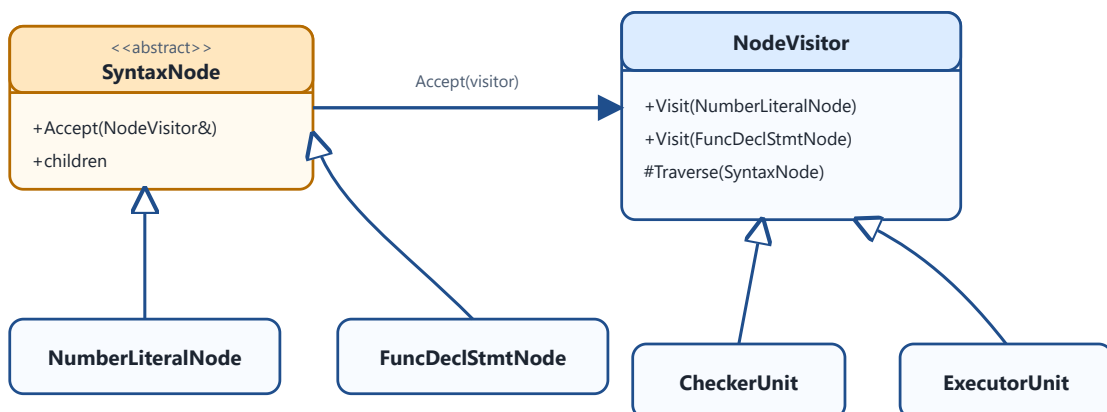
Composite Pattern



AST를 공통 노드 기반 트리 구조로 표현합니다.

Visitor

Visitor Pattern



AST 구조와 AST에 대한 동작을 분리합니다.

Singleton

Singleton Pattern



parser registry를 전역에서 하나만 유지합니다.

9. 클래스 상호 관계

Source Code

```

-> Tokenizer
-> TokenList
-> AssemblerUnit
    -> StatementParserRegistry
    -> IStatementParser implementations
    -> SyntaxNode AST
-> CheckerUnit
  
```

```
-> Static Scope Stack  
-> AST Annotation  
-> ExecutorUnit  
-> Runtime Scope Stack  
-> Value_t / Runtime Objects
```

10. 현재 설계의 한계

ExpressionParser 역할 혼합

expression parser와 expression statement parser 역할이 섞여 세미 콜론 처리 책임이 흐려집니다.

children index contract

노드별 children 순서가 암묵적이라 Parser, Checker, Executor가 같은 convention을 알아야 합니다.

ExecutorUnit 책임 집중

runtime scope, value, function, class, array 처리가 한 클래스에 모여 있습니다.

Singleton Registry

전역 registry는 테스트 간 상태 오염과 등록 순서 문제를 만들 수 있습니다.

Import 실행부 미구현

Parser/CheckerUnit은 ImportStmtNode를 실제로 처리하지만, ExecutorUnit에 해당 Visit()이 없어 alias 네임스페이스

DebugMode 스텝

DebugMode::Run()은 안내 메시지만 출력하며 step/next/break/watch/inspect가 전혀 구현되어 있지 않습니다.

(`math.add(...)`)가 아직 없습니다.
실제 코드 유입은 Tokenizer의 텍스트 splice에만 의존합니다.

11. 향후 리팩토링 방향

1

ExpressionStatementParser 분리

2

AST node shape table 문서화

3

Environment / ScopeStack 분리

4

FunctionRuntime, ClassRuntime, ModuleLoader 분리

5

StatementParserRegistry 테스트 안정성 개선

6

ParseError, SemanticError, RuntimeError 타입 계층 도입

7

Import ModuleLoader/alias 네임스페이스 실행부 구현
(ExecutorUnit::Visit(ImportStmtNode))

8

DebugMode step/next/break/watch/inspect 구현

12. 결론

CodeFab는 작은 인터프리터지만 compiler/interpreter에서 자주 쓰는 구조를 이미 갖추고 있습니다. Pipeline, Registry, Strategy, Composite, Visitor 패턴을 통해 기능 확장 기반을 만들었고, 이후 책임 분리를 강화하면 더 안정적인 구조로 발전할 수 있습니다.

현재 강점

단계별 파이프라인과 Visitor 기반 pass 분리.

핵심 개선점

ExecutorUnit 책임 집중과
ExpressionParser 역할 혼합 해소.